



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2233 — PROGRAMACIÓN AVANZADA 2026-1

Examen

30 de Junio 2026

Instrucciones

- La evaluación consta de dos partes: 10 preguntas de verdadero y falso, y 2 preguntas de desarrollo.
- Debes completar con tus datos personales en cada hoja utilizada. Luego debes entregar de vuelta **todas** las hojas de respuesta (tanto para verdadero y falso como de desarrollo) de la prueba, aunque estas se encuentren en blanco.
- Solo podrás retirarte de la sala una vez hayan transcurrido 60 minutos desde que inició de la evaluación.
- Durante la evaluación se realizarán 2 rondas de preguntas. Estas preguntas únicamente serán respondidas por los profesores del ramo.
- En caso de que sea necesario, podrás solicitar hojas blancas para apoyar al desarrollo de la prueba. Para esto levanta la mano y espera que un ayudante se acerque a tu puesto.

Verdadero o Falso [25 % nota final]

Indique en la hoja de respuestas la veracidad o falsedad de las siguientes 9 afirmaciones.

Independientemente de lo seleccionado, debe justificar su elección en un máximo de 4 líneas, el no justificar otorgará puntaje nulo en la pregunta.

Todas las afirmaciones tienen el mismo puntaje.

1. Suponiendo un computador donde funciona correctamente el comando “`python3 codigo/main.py`”. Si el archivo “`main.py`” llama a la función “`open("archivo.txt", "w")`”, esto escribirá en el archivo “`archivo.txt`” dentro de la carpeta “`codigo`”.
2. Se tiene una carpeta que contiene distintos módulos para testear un código. Para ejecutar un test en específico (`def test_probando_funcionalidad_1(...)`) basta con ejecutar el siguiente comando en consola:

```
python3 -m unittest test_probando_funcionalidad_1
```
3. Al utilizar *backtracking* para encontrar la solución de un tablero de Sudoku, el revertir los cambios realizados en una iteración o utilizar copias de los datos originales permite que el algoritmo pueda encontrar una solución.
4. Es posible implementar en Python un Iterable sin implementar un Iterador.
5. Si tengo un generador con *floats* correspondientes a las notas del examen. Puedo calcular el promedio de las notas utilizando únicamente la función `reduce()`, una *lambda function* y el generador mencionado, sin depender de otras funciones, clases o estructuras de datos.
6. Se puede utilizar una única *regex* para transformar un *string* en una lista de valores. Por ejemplo:
 - *String* inicial: `'Luna;Pepa,Chase/Guillermo!Rigoberto'`
 - Lista resultante: `['Luna', 'Pepa', 'Chase', 'Guillermo', 'Rigoberto']`
7. Es posible representar un grafo como una lista de adyacencia utilizando únicamente un diccionario y múltiples *sets*.
8. Es posible utilizar los algoritmos de DFS y BFS para recorrer una Lista Ligada.
9. Es posible crear un `array` de `numpy` que contenga todos los elementos de una lista con distintos tipos de datos. Por ejemplo: `[0, 'a', 1.0, True]`.

Preguntas Desarrollo

Pregunta 1. [50 % nota final]

Estás programando un juego online llamado **DCCarting** utilizando Python3, y PyQt6 para interfaces gráficas. En el juego, múltiples jugadores se conectan a jugar al mismo tiempo y compiten en una carrera de *karting*. El juego tiene las siguientes características:

- Se conectan 8 jugadores donde cada jugador tiene un cliente. Todos ellos se conectan a un mismo servidor. Una vez que se conectan los 8 jugadores, inicia un *timer* de 10 segundos y parte la carrera.
- Durante la carrera los clientes envían mensajes al servidor cada milisegundo para reportar la posición y velocidad de su vehículo en la carrera.
- El servidor recibe la información de cada jugador y mantiene un estado actual de la carrera que tiene la información más actualizada de la posición y velocidad de cada jugador.
- Una vez por milisegundo, el servidor envía a todos los clientes el estado actual de la carrera. Los clientes actualizan la posición de sus oponentes en el juego local basados en la información recibida por el servidor.
- Cada cierto tiempo, el servidor avisa a los clientes que hay disponibles 7 *bonificadores de velocidad* (1 jugador se quedará sin bonificador). Los jugadores reciben una notificación en el juego y deben apretar la tecla B del teclado para pedir el bonificador al servidor antes de que se acaben. A los primeros 7 jugadores que soliciten el bonificador se les entrega y aumenta su velocidad $\times 2$ durante 10 segundos.

Consideraciones:

- Supón que el juego es bastante simple y no hay choques entre los jugadores u otro tipo de interacción en el juego entre ellos.
- La modelación debe ser consistente entre preguntas.
- Puedes hacer supuestos razonables que no contradigan lo indicado en el enunciado.
- Todo *thread* que se cree debe implementarse como una clase (subclase de **Thread**); no se permiten *threads* basados en funciones (**Thread(target=...)**).

A partir de lo anterior, responde las siguientes preguntas:

1. **Modelación:**

- 1.1. **(2.5 puntos)** ¿Cuáles son las clases necesarias para modelar este juego en el cliente y en el servidor? Para cada clase describe en palabras: la lógica de la cual se encarga y si pertenecen al cliente o al servidor. Enfócate en las clases relacionadas a los elementos del juego descritos anteriormente y en los necesarios para la comunicación entre ellos.
 - 1.2. **(1 punto)** ¿Cuáles de las clases mencionadas anteriormente deben ser *threads*?
 - 1.3. **(1 punto)** ¿Cómo se comunican las clases mencionadas? Especifica las señales de PyQt, cualquier comunicación por la red y eventos de *threads* o variables compartidas.
2. **(2 puntos)** Utilizando pseudocódigo escribe el proceso del servidor desde que empieza a recibir conexiones de los clientes hasta el inicio de la carrera (los autos se empiezan a mover).

3. **Bonificadores de velocidad:**

- 3.1. **(2 puntos)** Desde el punto de vista de un cliente, describe en palabras los pasos necesarios para completar el proceso de *bonificadores de velocidad*. Inicia desde que el cliente recibe un mensaje que hay *bonificadores* disponibles, hasta que recibe una respuesta del servidor si consiguió el bonificador. Enfócate en describir la comunicación entre clases y la lógica que ejecuta cada clase involucrada.
 - 3.2. **(2 puntos)** Utilizando pseudocódigo escribe el método del servidor que maneja las solicitudes de los jugadores para obtener un bonificador de velocidad.
4. **(1.5 punto)** ¿Qué protocolo(s) de comunicación de red usarías para cada tipo de mensaje que intercambian los clientes y el servidor durante la carrera? Justifica tu decisión.

Pregunta 2. [25 % nota final]

DCCReservas es una aplicación cliente-servidor que permite a estudiantes, ayudantes y profesores gestionar la reserva de salas y laboratorios. Cada vez que un usuario realiza una acción desde la aplicación, el cliente debe enviar una solicitud al servidor. Esta solicitud contiene la siguiente información:

- **username** (**str**): nombre del usuario que realiza la acción.
- **accion** (**str**): acción solicitada (por ejemplo: crear una reserva, cancelar una reserva o consultar la disponibilidad de una sala).
- **datos** (**dict**): información asociada a la acción. Por ejemplo, para crear una reserva podría contener la sala, la fecha y el bloque horario solicitado. El contenido de este diccionario dependerá de la acción realizada.

La comunicación entre el cliente y el servidor se realiza mediante **sockets TCP**. Por esta razón, toda la información debe transformarse a una secuencia de *bytes* antes de ser enviada.

Además, como los campos **username**, **accion** y **datos** pueden tener largos variables, el servidor necesita saber donde comienza y donde termina cada parte del mensaje. Para esto, el mensaje debe incluir un **encabezado** que indique la información necesaria para reconstruir correctamente la solicitud recibida.

A partir de lo anterior, responde las siguientes preguntas:

- (1 puntos)** Diseñe un protocolo de serialización para transmitir la información descrita anteriormente. Debe indicar claramente la estructura del encabezado y del resto del mensaje.
- (2.5 puntos)** Escriba código en **Python** para implementar la función **serializar** de acuerdo con el protocolo diseñado. Puede asumir que los datos recibidos no están corruptos.
- (2.5 puntos)** Escriba código en **Python** para implementar la función **deserializar** de acuerdo con el protocolo diseñado. Puede asumir que los datos recibidos no están corruptos.